



Caches

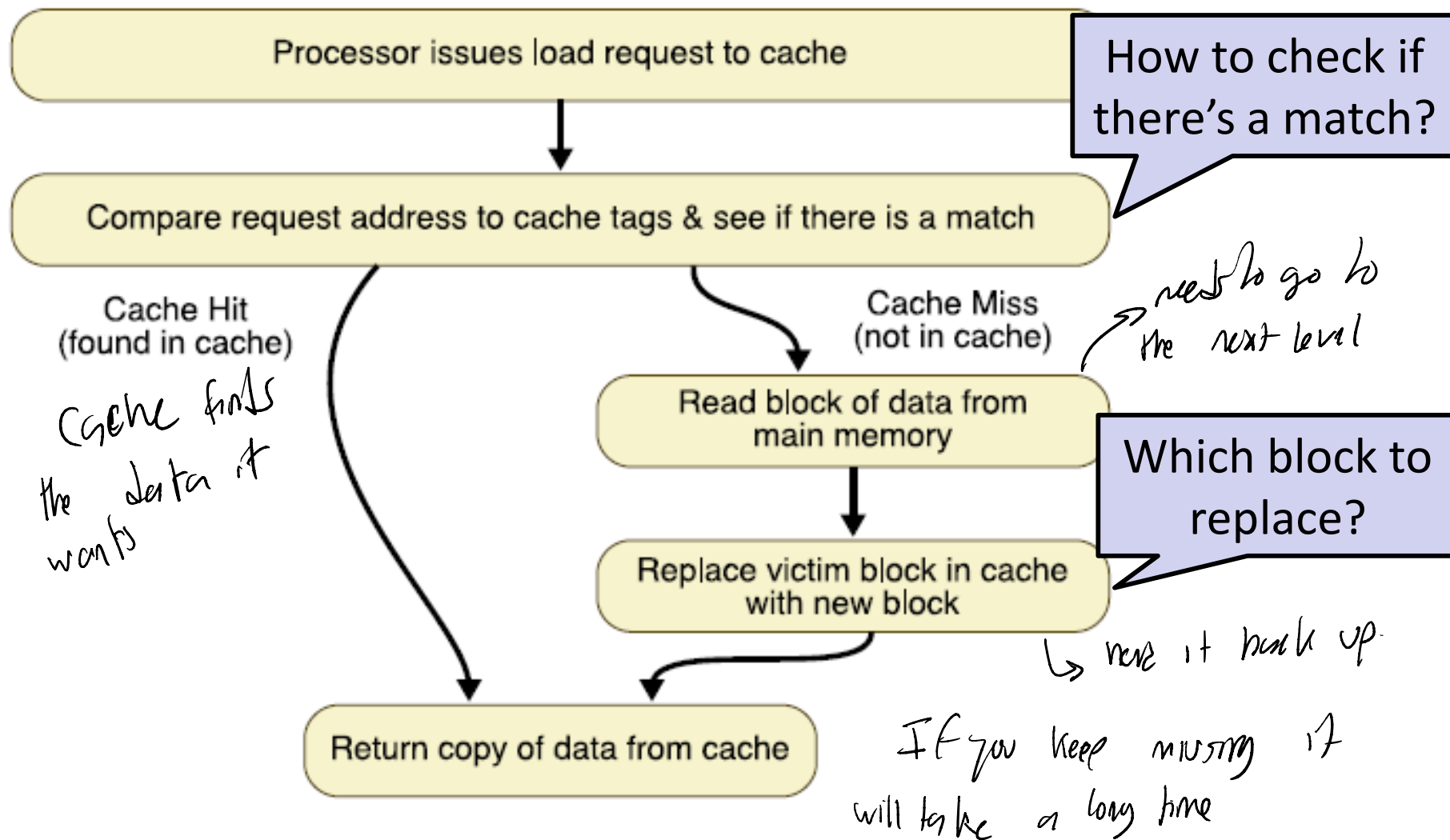
CS144

Prof. Yanjing Li

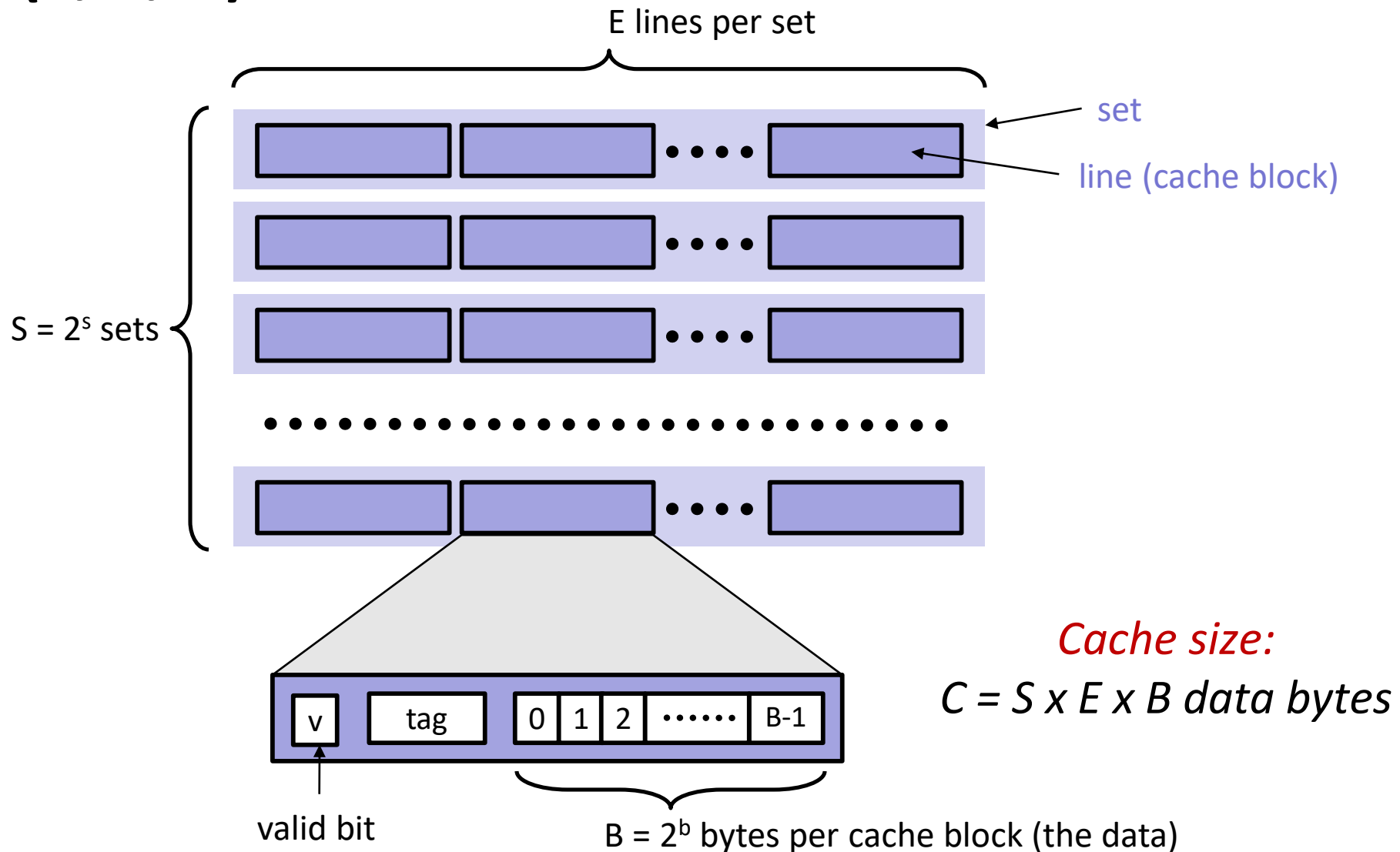
Memory Hierarchy, Locality

- Tradeoff: large and slow memory vs. small and fast memory
- The system has a hierarchy of memories 
- The memory hierarchy is only useful if the data/instructions that a program needs at a given point in time are already in the fast memories
- Why are data/instructions needed by the program in the fast memories (caches)?
- Locality
- accessing that data a lot*
- Leveraging the temporal locality: when a program access a data/instruction, put that in a cache
- Leveraging the spatial locality: when a program access a data/instruction, also put their neighbors in a cache
- close neighbors get stored*

Basic Flow for a “Mem $\rightarrow$ Reg” `mov`



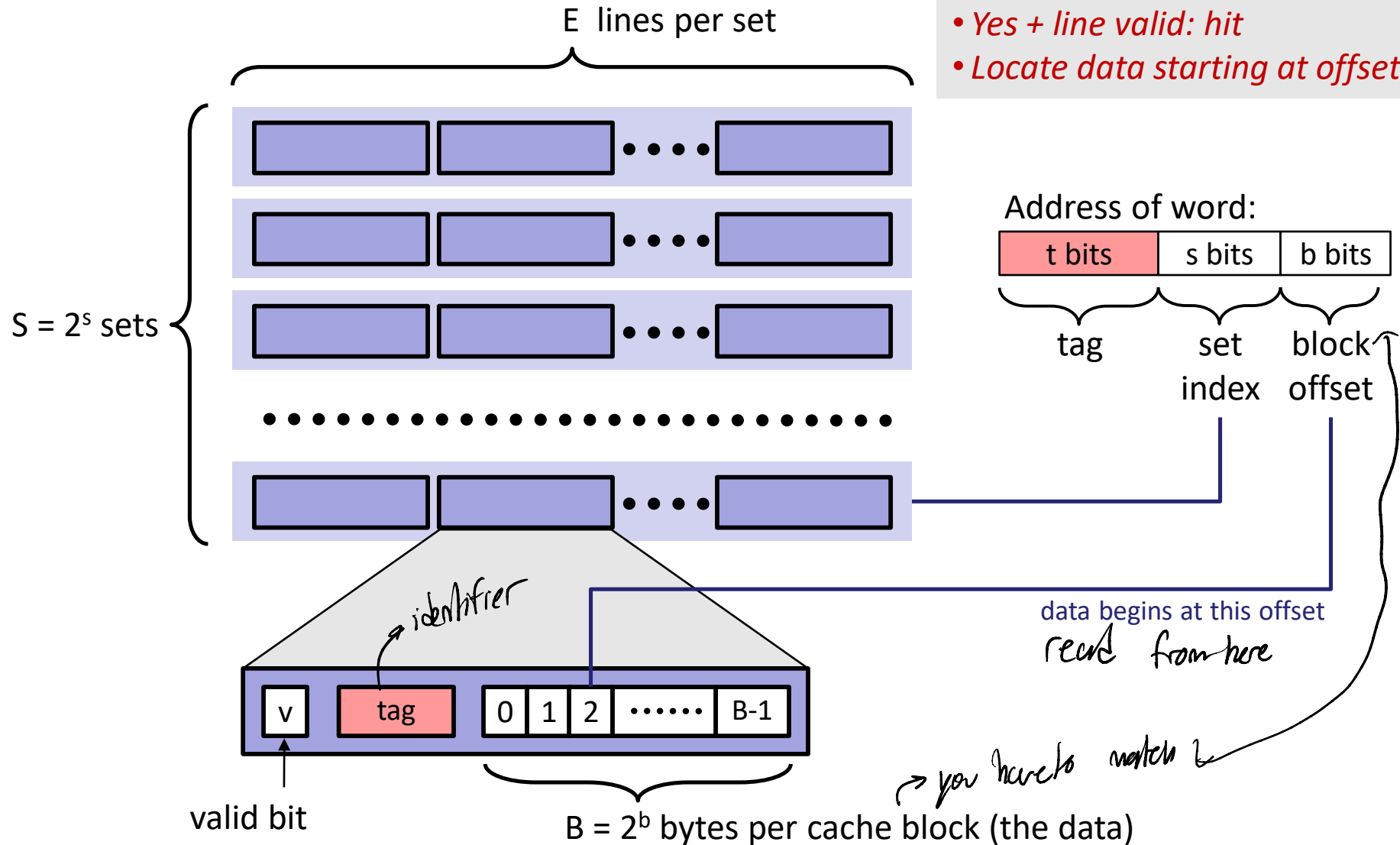
General Cache Design and Organization (S, E, B)





Cache Read: How Do I Find My Data?

- *Locate set*
- *Check if any line in set has matching tag*
- *Yes + line valid: hit*
- *Locate data starting at offset*

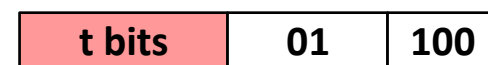
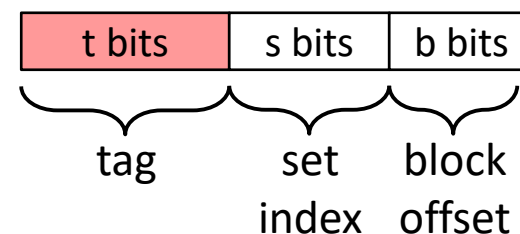


Example: Direct mapped cache ($E = 1$)

Direct mapped: One block per set

Assume: cache block size 8 bytes ($B=8$, $b=3$)

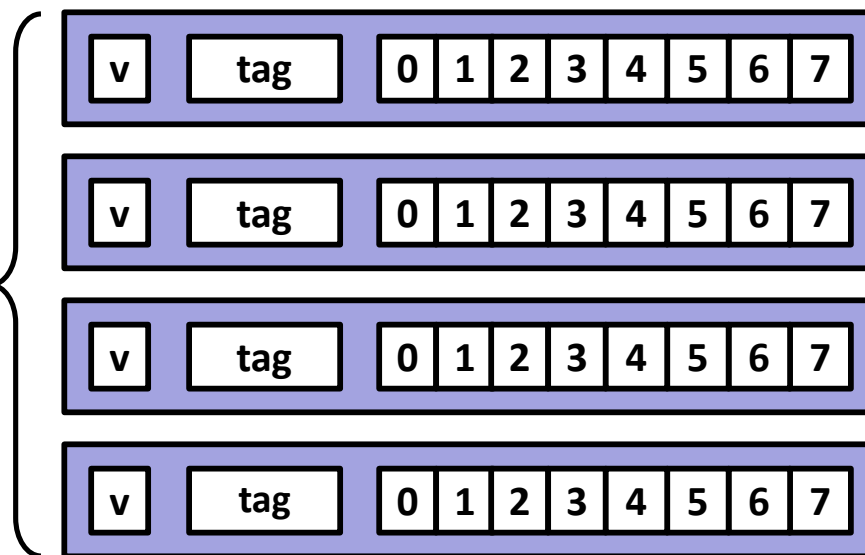
Address of data:



find set

$b = 3$
 $\downarrow$
 $2^3 = 8$

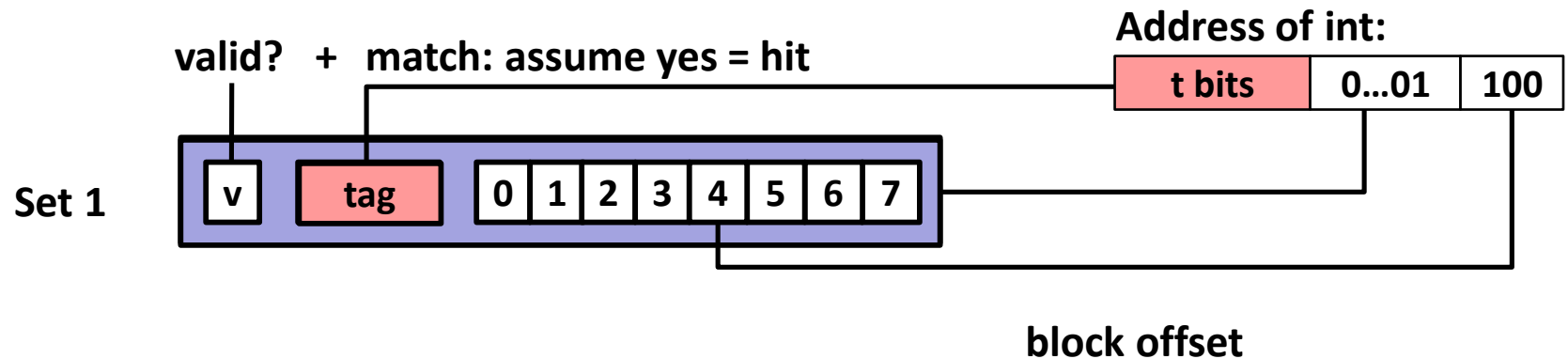
$S = 2^s$ sets
 $(s=2)$



Example: Direct mapped cache (E = 1)

Direct mapped: One block per set

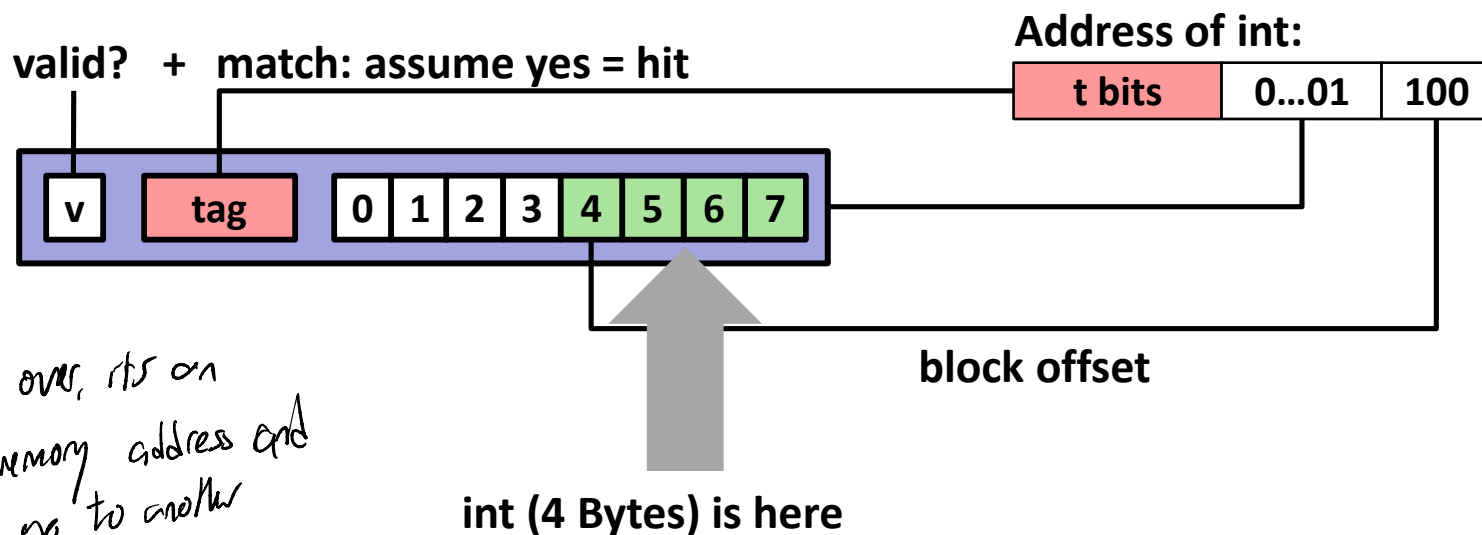
Assume: cache block size 8 bytes



Example: Direct mapped cache (E = 1)

Direct mapped: One block per set

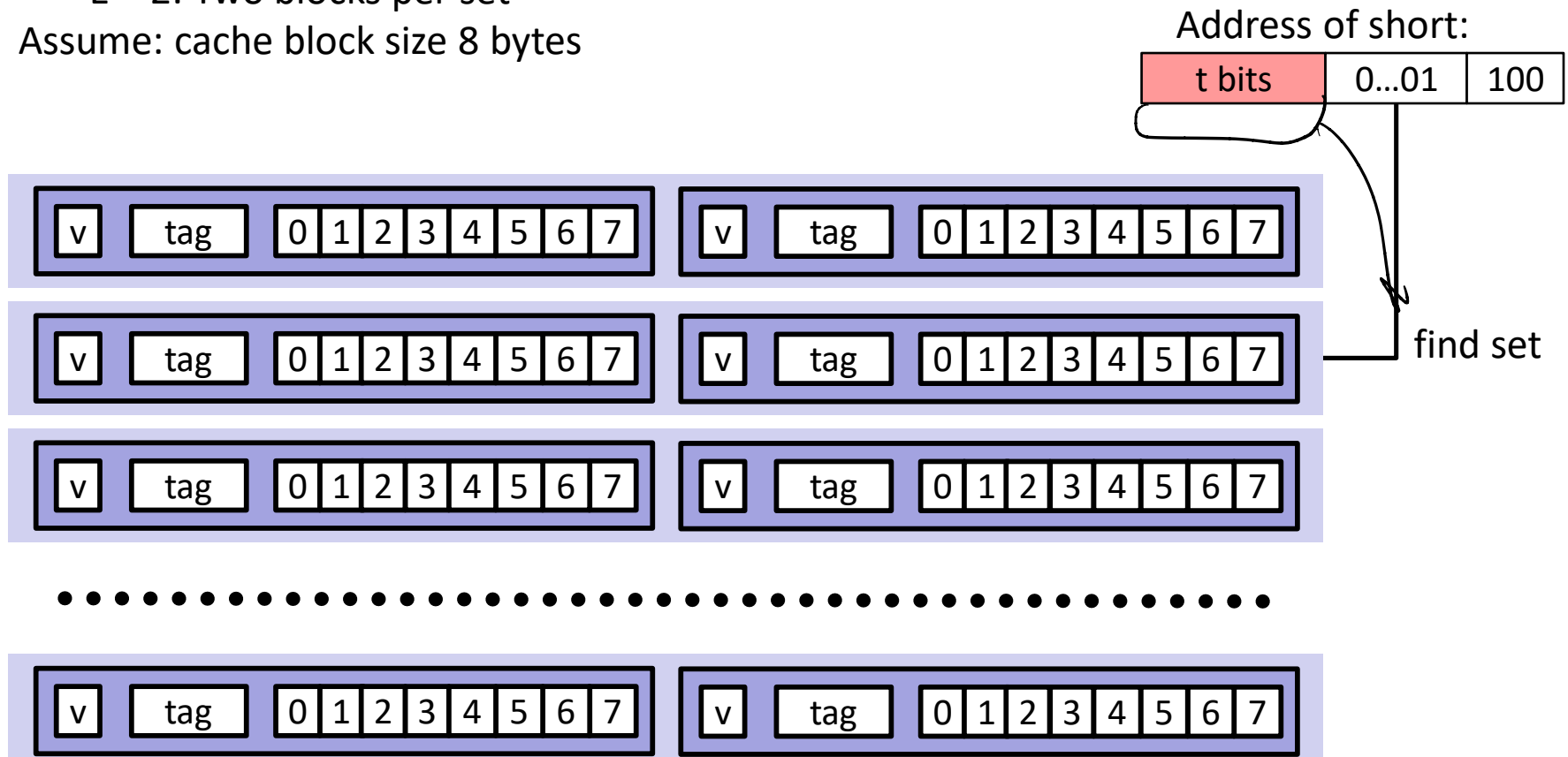
Assume: cache block size 8 bytes



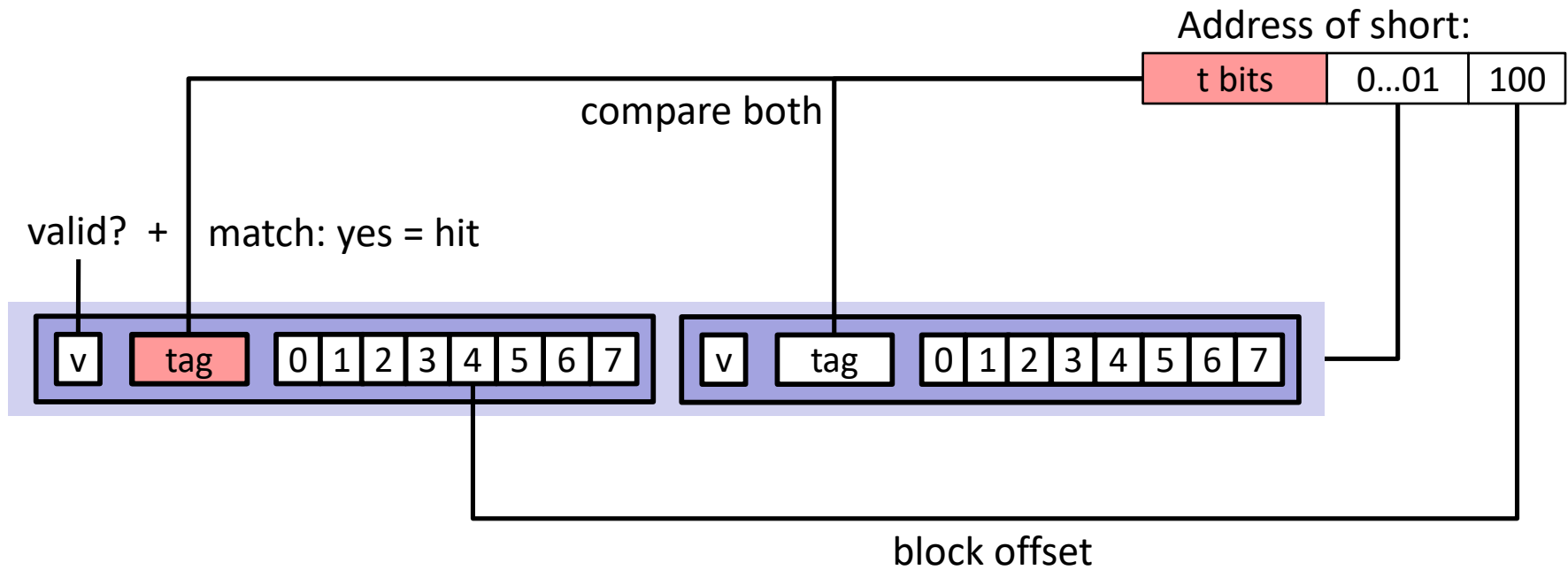
No match: old block is evicted and replaced

Example: 2-way set associative cache

$E = 2$: Two blocks per set
Assume: cache block size 8 bytes

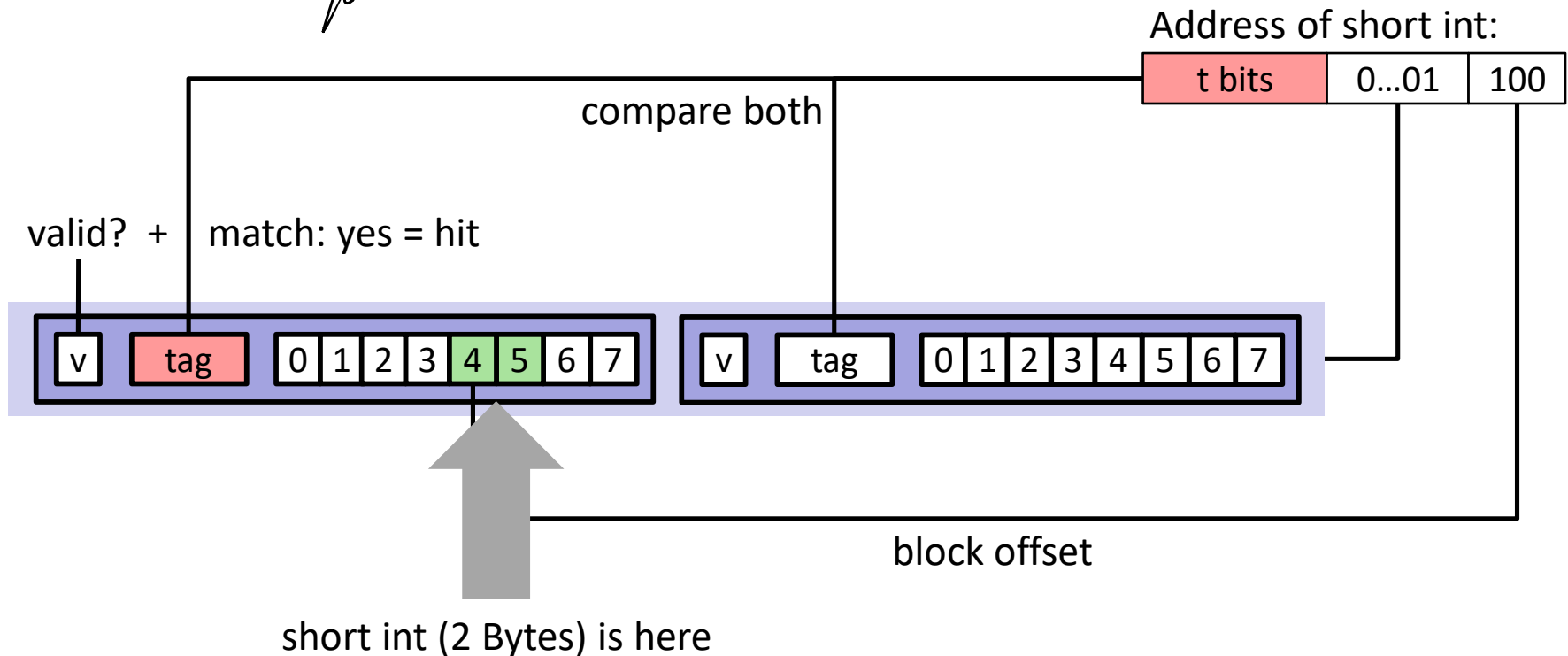


Example: 2-way set associative cache



Example: 2-way set associative cache

you need to check 2



No match:

- One block in set is selected for eviction and replacement

Categorizing Misses: The Three C's

Compulsory (or cold) – first-reference to a block, occur even with infinite cache
→ you have not seen the data before

Capacity – cache is too small to hold all data needed by program at a given time (refer to as the working set)
happens when you need more data

Conflict – misses that occur because of collisions, even when the cache is large enough

E.g., referencing blocks 0, 8, 0, 8, 0, 8 ... would always miss if 0 and 8 are mapped to the same cache line in a large direct-mapped cache

*if you have
two way set,
you won't kick each
other out*

*because these addresses reference the same block,
you'll just keep kicking the last one out*

Direct-mapped cache simulation

t=1	s=2	b=1
x	xx	x

M=4 bit addresses, B=2 bytes/block,
S=4 sets, E=1 Block/set

Address trace (reads, one byte per read):

Spatial
Locality

since they are close

Temporal
Locality
(Unexploited)

when it's referenced a couple of times

0	[0000] ₂ ,	miss [cold]
1	[0001] ₂ ,	hit
7	[0111] ₂ ,	miss [cold]
8	[1000] ₂ ,	miss [cold]
0	[0000] ₂ ,	miss [conflict]

we've never seen it before

leverages spatial locality

cuz we haven't seen this data

	v	Tag	Block
Set 0	1	0	M[0-1]
Set 1			
Set 2			
Set 3	1	0	M[6-7]

2-way set associative cache simulation

t=2	s=1	b=1
xx	x	x

M=4-bit addresses, B=2 bytes/block,
S=2 sets, E=2 blocks/set

Address trace (reads, one byte per read):

Spatial
Locality

0	[0000] ₂ ,	miss <i>cold</i>
1	[0001] ₂ ,	hit
7	[0111] ₂ ,	miss <i>cold</i>
8	[1000] ₂ ,	miss <i>cold</i>
0	[0000] ₂	hit

Temporal
Locality
(Exploited)

	v	Tag	Block
Set 0	1	00	M[0-1]
	1	10	M[8-9]
Set 1	1	01	M[6-7]
	0		

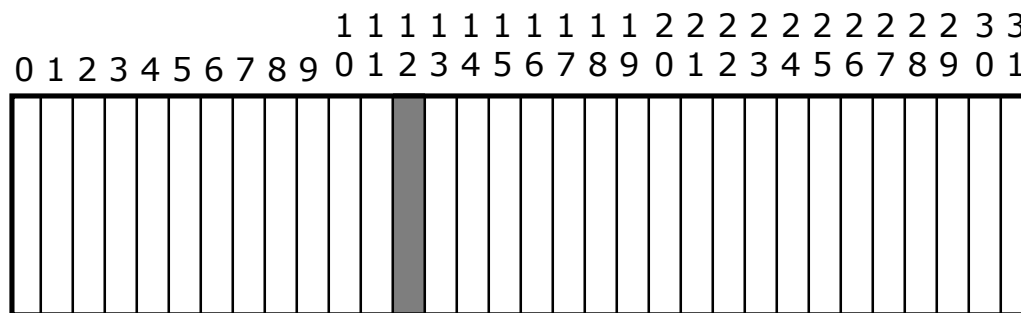
*No conflict
misses!*



Set Associativity Affects Data Placement

Block number

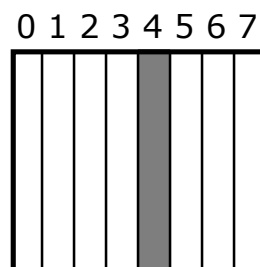
Memory



Where can block 12 be placed?

Set Number

Cache



Direct
Mapped

*direct mapping will have a
conflict every 8*

Assume: each cache block is 8 bytes
total memory size = 256 bytes

Memory is divided into $256/8$ 32 cache blocks

Address of data in block 12:

01	100	xxx
----	-----	-----

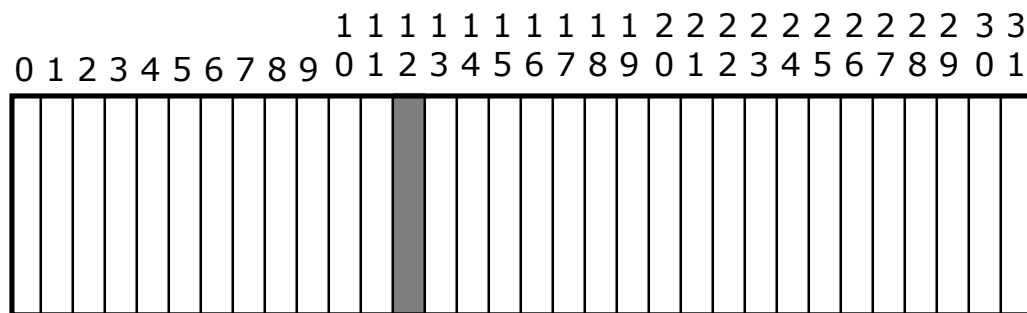
tag Set
index Block
offset



Set Associativity Affects Data Placement

Block number

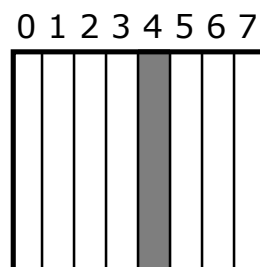
Memory



Where can block 12 be placed?

Set Number

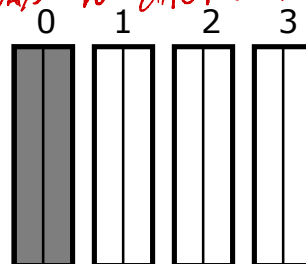
Cache



Direct
Mapped

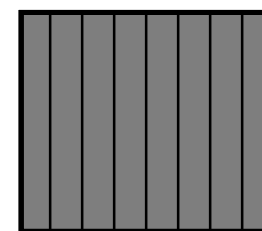
Only into
set 4
(12 mod 8)

*now 12 can
map to another block*



2-way set
associative

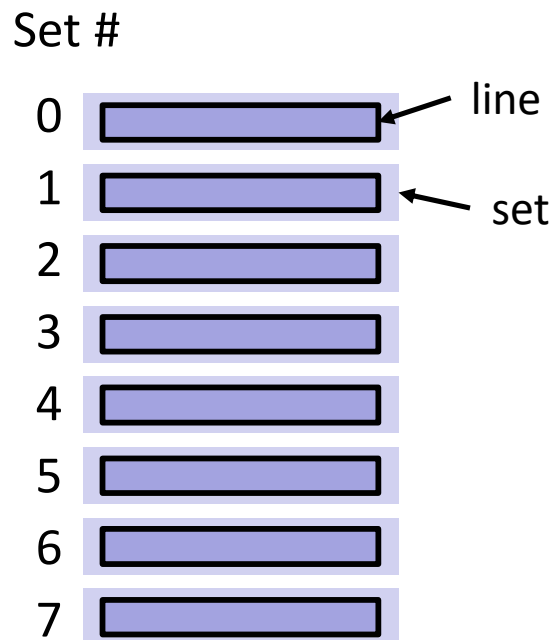
Anywhere
in set 0
(12 mod 4)



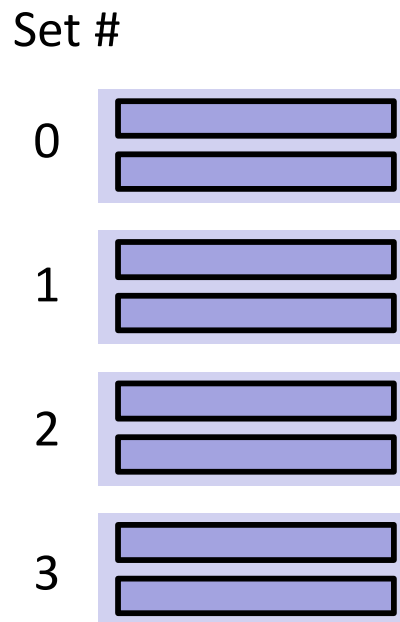
fully
associative

Anywhere

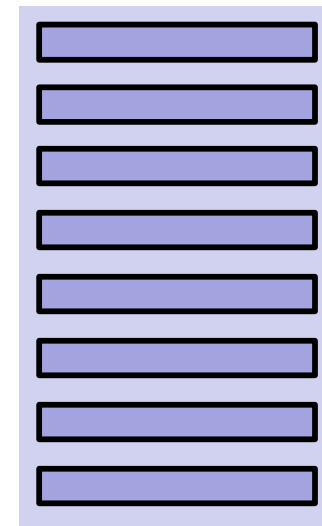
Associativity vs. Cache Misses



**Direct
Mapped**



**2-way set
associative**



**fully
associative**

More cache misses
Simpler lookups



Less cache misses
More complex lookups

Other Basic Cache Design Considerations

Block Replacement:

Which block should be replaced on a miss?

Write Strategy:

What happens on a write to memory (e.g., `mov data from reg to mem`)?

Block Replacement: Which block to vote off?

No choice in a direct mapped cache

In an associative cache, which block from set should be evicted when the set becomes full?

Random

Least Recently Used (LRU)

LRU cache state must be updated on every access

First In, First Out (FIFO) aka Round-Robin

Used in highly associative caches

Write Strategy: How are writes handled?

Cache Hit

Write Through – write to memory immediately, generally higher traffic but simpler to design

Write Back – write cache only, memory is written when evicted, dirty bit per block avoids unnecessary write backs, more complicated

Cache Miss

No Write Allocate – only write to main memory

Write Allocate – fetch block into cache, then write

Common Combinations

Write Through & No Write Allocate (old)

Write Back & Write Allocate (new)

Cache Performance Metrics

Miss Rate

Fraction of memory references not found in cache (misses / accesses)
 $= 1 - \text{hit rate}$

Typical numbers (in percentages):

3-10% for L1

can be quite small (e.g., < 1%) for L2, depending on size, etc.

Hit Time

Time to deliver a line in the cache to the processor
includes time to determine whether the line is in the cache

Typical numbers:

1-4 clock cycles for L1

10 clock cycles for L2

Miss Penalty

Additional time required because of a miss
typically 50-200 cycles for main memory (Trend: increasing!)

Let's think about those numbers

Huge difference between a hit and a miss

Could be 100x, if just one level cache and main memory

Would you believe 99% hits is twice as good as 97%?

Consider:

cache hit time of 1 cycle

miss penalty of 100 cycles

Know this
↓

Average memory access time = $\text{hit time} + \text{miss rate} * \text{miss penalty}$ (for one level cache)

97% hits: $1 \text{ cycle} + 0.03 * 100 \text{ cycles} = 4 \text{ cycles}$

99% hits: $1 \text{ cycle} + 0.01 * 100 \text{ cycles} = 2 \text{ cycles}$

Misses in caches are inevitable. But.. Good programmers can optimize code to improve cache performance (next time).